

CEG 4166 / CSI 4141 – Real-Time Systems Design
SEG 4145 – Real-Time and Embedded Software Design
Winter 2026

Laboratory 4 – Speed Measurement and Closed-Loop Control

Extension of the previous motor-control laboratories

Period: March 14 → March 28

Weight: 20%

1. Context and Pedagogical Continuity

Laboratory 4 addresses encoder-based speed measurement and closed-loop speed regulation for the DC motor integrated into the STM32 Nucleo L552ZE-Q Platform. The main objective of this laboratory is to extend the previous open-loop PWM motor-control implementation into a deterministic feedback-control system capable of measuring the actual motor speed, comparing it with a reference value, and automatically adjusting the PWM duty cycle in real time.

This laboratory also incorporates user interaction and timing analysis. Students are required to display the measured motor speed on the LCD, allow the user to select predefined speed references through the keypad, and provide audible feedback using the buzzer. The overall design must remain modular, compatible with RTOS-based development, and scalable for subsequent control-oriented laboratory work.

In addition, Lab 4 implementation must comply with a modular software design approach. Students are expected to organize their code into separate source and header files according to functionality. For example, the project should include dedicated modules such as keypad.c / keypad.h, motor.c / motor.h, lcd.c / lcd.h, along with other relevant modules for encoder acquisition, buzzer management, and timing services.

Single-file or monolithic implementations are not acceptable. This requirement is intended to promote software clarity, maintainability, reusability, and ease of integration in an embedded real-time system.

Super-loop implementations are not permitted. The software architecture developed in the previous motor-control laboratories must be reused, extended, and properly adapted to meet the new closed-loop control requirements, rather than being rewritten from scratch...

2. Learning Objectives

By the end of this laboratory, students will be able to:

- Measure motor speed using the quadrature encoder signals
- Convert encoder events into deterministic RPM estimates
- Implement a closed-loop speed controller using automatic PWM adjustment
- Maintain deterministic behavior under periodic execution constraints
- Integrate LCD output, keypad presets, and buzzer feedback into a modular embedded architecture
- Perform basic timing and jitter analysis on the control loop

3. Hardware Configuration (Mandatory)

The following hardware connections shall be used for Laboratory 4. These signal assignments are based on the official CEG3136 lab user guide for the STM32L552ZE-Q platform.

3.1 Motor Driver

Function	Pin
Standby (STB)	PE15
PWM (TIM1_CH1)	PE9
AI1	PB10
AI2	PB11

3.2 Encoder Inputs

Function	Pin
Encoder channel A	PB0 (EXTI)
Encoder channel B	PB1 (EXTI)

The motor encoder provides two feedback channels. The guide specifies 11 pulses per revolution at the motor shaft, a 34:1 gear ratio, and therefore 374 pulses per revolution at the geared output shaft for each encoder channel. The rated no-load motor speed is 350 RPM at 12 V.

3.3 LCD Display

Function	Pin
I2C SDA	PF0
I2C SCL	PF1

The LCD shall display the target speed preset and the measured speed in RPM. Students may reuse the LCD driver previously provided in the course, or an equivalent modular LCD driver approved by the instructor.

3.4 Keypad / Touch Input for Speed Presets

Function	Pin
Touch interrupt	PB6
Touch enable	PB5
I2C SDA	PF0
I2C SCL	PF1

3.5 Buzzer

Function	Pin
Buzzer	PA0

The buzzer implementation must be non-blocking.

4. System Behavior Requirements

4.1 Initial State (Mandatory)

At reset:

- Motor state = STOP
- PWM duty cycle = 0%
- Closed-loop controller inactive until a valid preset is selected
- LCD displays STOP or 0 RPM depending on the initialization strategy chosen
- Motor not rotating

4.2 Speed Measurement Requirement

The system must compute the actual motor speed from the encoder signals. Students may use edge counting over a fixed sampling window, input capture timing, or another deterministic method approved by the instructor.

- The measured speed must be expressed in revolutions per minute (RPM).
- The speed computation must be updated periodically.
- The method must be deterministic and clearly documented.
- Encoder handling must remain robust under rapid speed changes.

4.3 Closed-Loop Control Requirement

The motor speed must be regulated automatically. The controller shall compare the measured RPM with the selected target RPM and adjust the PWM duty cycle accordingly.

- Open-loop fixed-PWM solutions are not acceptable.
- The PWM update must be performed automatically by software based on the speed error.
- The control law may be proportional or proportional-integral, provided that the behavior is stable and justified.
- The motor must remain safe and predictable when the target speed changes.

4.4 Keypad Preset Mapping (Mandatory)

The keypad/touch interface must be used to select closed-loop speed presets. The following mapping is mandatory for Laboratory 4:

Input	Function	Feedback
Key "1"	Low speed preset	1 beep
Key "2"	Medium speed preset	2 beeps
Key "3"	High speed preset	3 beeps
Key "0"	STOP	Motor off

Preset values in RPM shall be defined by the student and documented in the report. The three presets must be clearly separated and experimentally reachable on the real hardware.

Each valid keypad command must:

- Update the target speed reference
- Trigger the appropriate buzzer pattern
- Update the LCD immediately
- Preserve deterministic system behavior

4.5 LCD Display Requirements

The LCD must display meaningful real-time information about the control loop. At minimum, it must show the measured speed in RPM. The recommended display format is shown below.

LCD Line	Displayed Information
Line 1	Preset / target speed
Line 2	Measured speed in RPM

Integration rules:

1. LCD initialization must be completed once at startup before normal task execution begins.
2. LCD updates must be handled inside a task, not inside an ISR.
3. The displayed RPM value must follow the measured motor speed.
4. The display must be refreshed after every valid preset change.
5. No blocking delay shall be introduced solely for LCD handling.

4.6 Buzzer Behavior

The buzzer is used as user feedback for preset selection. The buzzer must operate without blocking the control loop or the scheduler.

- 1 beep = low speed preset
- 2 beeps = medium speed preset
- 3 beeps = high speed preset
- Optional short confirmation tone for STOP

HAL_Delay() inside the control logic is forbidden.

5. Required Software Architecture

Students must implement a structured RTOS-based design. Exact task names may vary, but the separation of responsibilities must be clear.

5.1 Motor / Control Task (Highest Priority)

Responsible for:

- Reading the latest speed estimate or consuming it from a shared interface
- Applying the closed-loop control law
- Updating PWM duty cycle
- Enforcing STOP behavior and output safety

Recommended period: 10-20 ms

5.2 Encoder Measurement Task or Service Layer

Responsible for:

- Capturing encoder events
- Computing speed in RPM
- Publishing a deterministic speed estimate to the control logic

This functionality may be implemented using interrupts plus deferred processing, or a periodic task that reads encoder counters, provided the design remains deterministic.

5.3 User Interface Task

Responsible for:

- Processing keypad commands
- Updating target speed presets
- Refreshing the LCD
- Triggering buzzer patterns

5.4 Interrupt Service Routines

ISRs must remain short and deterministic.

- ISRs must not implement lengthy LCD transactions.
- ISRs must not contain blocking delays.
- ISRs should only capture events, set flags, notify tasks, or increment counters.
- Any non-trivial computation should be deferred to task level.

6. Determinism, Timing, and Jitter Analysis

Timing behavior is part of the laboratory evaluation. Students must analyze the periodic execution of the control loop and discuss whether the loop timing remains stable under keypad activity and LCD updates.

At minimum, the report must include:

- Chosen control-loop period
- Method used to measure execution timing or task release timing
- Observed jitter or timing variation
- Brief discussion of the impact of timing variation on speed regulation

7. Required In-Lab Demonstration

Students must demonstrate all of the following on the real platform:

6. Encoder-based speed measurement running in real time
7. Measured RPM displayed on the LCD
8. Closed-loop automatic PWM adjustment
9. Keypad selection of low, medium, and high speed presets
10. Buzzer patterns for each preset
11. Deterministic STOP behavior
12. A brief explanation of the timing and jitter analysis

8. Deliverables

8.1 Functional Implementation

- Closed-loop speed-control implementation
- Encoder-based speed measurement
- LCD speed display
- Keypad preset selection
- Buzzer preset feedback
- Clean modular source code

8.2 Short Technical Explanation

- How RPM is computed from the encoder
- How the controller adjusts PWM automatically
- How determinism is preserved in the software design

8.3 Timing and Jitter Analysis

- Measurement method
- Observed timing behavior
- Short interpretation of the results

9. Evaluation Rubric – Lab 4 (20 pts)

Criterion	Points
Encoder handling	5
Speed computation	4
PWM adjustment	4
LCD + keypad presets	3
Buzzer preset feedback	2
Code modularity	1
Timing analysis	1
Total	20

10. Instructor Note

This laboratory prepares students for more advanced feedback-control experiments in embedded systems, including tighter real-time constraints, improved controller design, and more formal validation of timing behavior.